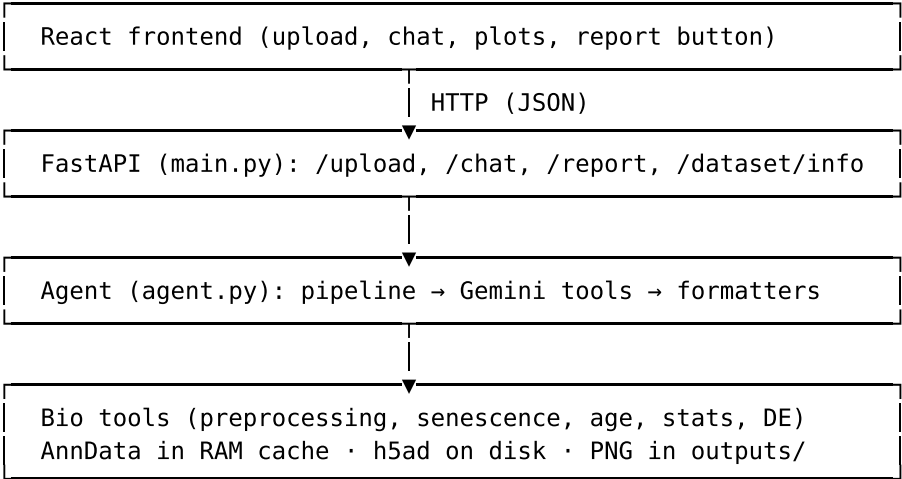


Version: May 2026
Stack: React frontend · FastAPI backend · Google Gemini (tool routing) · Scanpy (analysis)

1. Big picture: three layers

[code block]



Core design rule: Python owns all science; Gemini chooses which tool to call and may add prose. Numbers in chat usually come from Python `_format_*_summary` formatters reading tool JSON, not from free-form LLM invention.

2. The central object: AnnData (adata)

Everything scientific hangs off one in-memory adata per `file_id`:

Location	Contents
adata.X	Expression matrix after normalization (viz / clustering)
adata.layers["counts"]	Raw counts locked at pipeline start (for DESeq2)
adata.obs	Per-cell metadata: age, cell_ontology_class, leiden, senescence_score, ...
adata.obsm["X_umap"]	UMAP coordinates

```
adata.uns["pipeline_state"] | Flags: qc, norm, cluster (pipeline runs once)
adata.uns["metadata_status"] | Result of check_required_metadata
adata.uns["dataset_summary"] | Optional cached dataset context
```

Lifecycle:

1. Upload → disk: backend/data/uploads/{uuid}.h5ad
2. First chat → `sc.read_h5ad` → `cache_adata(file_id, adata)`
3. Later chats → `get_adata(file_id)` (TTL 1 hour, max 3 datasets in cache)
4. Same adata object is mutated in place (scores, clusters persist)

3. Frontend (frontend/src/)

App.tsx – orchestrator

- sessionId: UUID per browser session; new one on reset.
- fileId / fileName: from /upload response.
- species: "mouse" or "human" (locked after upload).
- history: {role, content}[] sent to backend every chat.
- sessionToolRuns: accumulates every tool_calls entry from /chat (used for reports).
- plots: {url, caption}[] – URLs like /plots/umap.png, displayed via API_BASE.

uploadFile: FormData → POST /upload → clears history, plots, tool runs.

sendMessage: POST /chat with session_history including current user message. Appends assistant reply; merges data.plots; extends sessionToolRuns.

generateReport: POST /report with tool_runs: sessionToolRuns (authoritative), not chat prose.

config.ts

API_BASE from VITE_API_URL or http://127.0.0.1:8000.

Plots.tsx

Builds image URLs as API_BASE + plot.url.

ChatPanel.tsx

Shows tool names from last turn; Report button uses accumulated tool run count.

4. API layer (backend/main.py)

Global state (in-process)

- sessions: backup chat history by session_id.
- session_tool_runs: append-only log of each tool call from /chat.

POST /upload

1. Reject non-.h5ad files.
2. file_id = uuid4().
3. Write to uploads dir via persist_upload.
4. Return {file_id, species}.

POST /chat

1. resolve_dataset_path(file_id).
2. Sanitize session_history roles.
3. run_agent(...) – heart of the app.
4. On success: append to sessions and session_tool_runs.
5. On exception: generic error reply (no stack trace to client).

Response shape:

[code block]

```
{
  "reply": "...",
  "plots": [{"url": "/plots/...", "caption": "..."}],
  "tool_calls": [{"name": "...", "args": {}, "result": {}}]
}
```

POST /report

1. Load tool_runs from body or session_tool_runs.
2. User questions only from history (role == user).
3. generate_report – Gemini writes Markdown from tool JSON log only.

4. save_report_files – writes .md + .pdf.

GET /dataset/{file_id}/info

Reads h5ad, build_dataset_summary, caches adata, returns JSON.

Static mounts

- /plots → backend/outputs/
- /reports → backend/outputs/reports/

5. Dataset paths (backend/dataset_paths.py)

Function | Behavior

----- | -----

ensure_uploads_dir() | Creates backend/data/uploads/

persist_upload(file_id, source) | shutil.copy2 to durable path

resolve_dataset_path(file_id) | Persistent file, else temp legacy path

6. In-memory cache (backend/agent/cache.py)

Function | Behavior

----- | -----

cache_adata(file_id, adata) | Store + timestamp; evict expired (>3600s) and oldest if >3

get_adata(file_id) | Return adata or None; refresh timestamp on hit

7. Deterministic pipeline (backend/agent/pipeline.py)

ensure_pipeline(adata, species) runs before any LLM tool. The model cannot skip or reorder these steps.

Step 0 – repair + metadata

- Delete uns["loglp"] if wrongly set to True (breaks Scanpy HVG).

- Load pipeline_state dict.
- Once: `adata.uns["metadata_status"] = check_required_metadata(adata).`

`check_required_metadata (tools/preprocessing.py)`

- Requires age, cell_ontology_class, sample_id in obs.
- Fallback: copy mouse.id / mouse_id / donor_id / batch → sample_id.
- Returns {status: "ok"} or {status: "degraded", missing: [...]}.

Step 1 – lock counts

If no `layers["counts"]`: copy `adata.X` to `layers["counts"]` (raw for pseudobulk).

Step 2 – QC (quality_control)

- `filter_cells` min 200 genes; `filter_genes` min 3 cells.
- Sets `qc_done = True`.

Step 3 – normalize

- `normalize_total + log1p` on `adata.X` (visualization only).
- Sets `senescence_agent_viz_normalized` (does NOT set `uns["log1p"] = True`).

Step 4 – cluster (`cluster_cells`)

- `highly_variable_genes` → `pca` → `neighbors` → `leiden` → `obs["leiden"]`.

8. Gene layer (backend/tools/gene_utils.py)

On import:

- Loads human SenMayo list from JSON.
- `normalize_gene_names` → mouse orthologs via MyGene API + fallbacks.
- Exposes `SENESCENCE_GENES` and `SENESCENCE_GENES_MOUSE`.

Species parameter selects which list tools use.

9. Tool router (backend/agent/tool_router.py)

build_tool_map(adata, species, tools) returns lambdas Gemini invokes:

Tool name	Python function
generate_umap	generate_umap(adata)
find_senescence_markers	find_senescence_markers(adata, species)
senescence_score	senescence_score(adata, species)
get_cluster_annotations	get_cluster_annotations(adata)
compare_across_age	compare_across_age(adata, ...)
test_senescence_difference	test_senescence_difference(adata, ...)
run_deseq2	run_deseq2_wrapper (pseudobulk + PyDESeq2)
run_deseq2_wrapper	

1. build_pseudobulk_matrix(adata, cell_type) – sum raw counts per sample.
2. run_deseq2_pseudobulk(count_df, meta_df, ...) – PyDESeq2.
3. Serialize top genes; attach group labels and sample counts.

10. Analysis tools (backend/tools/)

generate_umap (visualization.py)

- sc.tl.umap if missing; plot leiden; save outputs/umap.png.

find_senescence_markers (senescence.py)

- Intersect SenMayo list with var_names; return found/missing/coverage_pct.

senescence_score (senescence.py)

- sc.tl.score_genes → obs["senescence_score"].
- UMAP plot → senescence_score.png.
- Mean score per leiden cluster; dominant cell type per cluster.

get_cluster_annotations (senescence.py)

- Per cluster: dominant cell_ontology_class + distribution.

compare_across_age (age_analysis.py) – DESCRIPTIVE

- Median senescence_score by age (and optional cell_type filter).
- Optional reference_age vs comparison_age subset.
- Bar + violin plots to outputs/.
- Does NOT return sample-level p-values.

test_senescence_difference (statistics.py) – INFERENCE

- Filter one cell_type.
- Per sample (mouse): median senescence_score across its cells.
- Mann-Whitney U on sample medians (e.g. 4 vs 4 mice, not cell counts).
- Returns p_value, effect_size, inference_tier, warnings.

build_pseudobulk_matrix (build_pseudobulk.py)

- Filter cell type; sum layers["counts"] per sample_id → pseudobulk matrix.

run_deseq2_pseudobulk (run_deseq2.py)

- Youngest vs oldest age groups (or user contrast).
- PyDESeq2 on pseudobulk; FDR-adjusted gene table.

11. Gemini tool schema (backend/agent/tool_schema.py)

TOOLS = function_declarations with parameter names/descriptions. This is the only set of actions the model can invoke.

Seven tools: generate_umap, find_senescence_markers, senescence_score, get_cluster_annotations, run_deseq2, compare_across_age, test_senescence_difference.

12. System prompt (backend/agent/system_prompt.py)

Instructions: SenMayo definition, three analysis levels (cell / population / gene), rules (no fake numbers, use test_senescence_difference for p-values on scores, DESeq2 for gene-level DE).

On first message only: append format_dataset_context(build_dataset_summary(adata)).

13. Agent orchestration (backend/agent/agent.py)

Routing helpers

Function	Purpose
_agent_iteration_limit(message) | Default 3 Gemini rounds; 5 for full-panel phrases
_wants_analysis_panel(message) | "run everything", "what's interesting", etc.
_wants_multi_step(message) | Prevents early exit after one tool
_needs_pvalue_clarification(message) | Extra disclaimers for significance questions

run_analysis_panel

Bypasses Gemini. Fixed sequence: markers → score → umap → annotations → compare_across_age.

_collect_plots_from_result

Maps plot_path, age_distribution_plot, senescence_violin_plot to /plots/ URLs.

Direct formatters (anti-hallucination)

Formatter	Tool
_format_deseq2_summary | run_deseq2
_format_test_summary | test_senescence_difference
_format_age_summary | compare_across_age
_format_senescence_score_summary | senescence_score
_format_marker_summary | find_senescence_markers
_format_cluster_summary | get_cluster_annotations

If tools are in DIRECT_SUMMARY_TOOLS and not multi-step, return formatted Python text without a second Gemini narrative pass.

run_agent – main loop

1. get_adata or read_h5ad + cache_adata
2. ensure_pipeline(adata, species)
3. build_tool_map
4. IF analysis panel phrase → run_analysis_panel (no Gemini)
5. GenerativeModel + start_chat(history)
6. FOR each iteration (max 3–5):
 - send_message
 - IF no function call → return text + plots + tool log
 - Execute each function call via tool_map
 - IF direct summary tools only → return formatted summaries
 - ELSE send function responses back to Gemini for another round
7. IF exhausted → iteration limit message

14. Reports (backend/agent/report.py)

Function | Role

----- | -----

_sanitize_result_for_report | Truncate DE genes; basename plot paths

_format_tool_runs | Markdown sections with JSON args + results

_compact_user_questions | User messages only

generate_report | Gemini writes from tool log only

save_report_files | MD + PDF via matplotlib PdfPages

Reports intentionally ignore assistant chat as a data source.

15. Dataset info (backend/tools/dataset_info.py)

build_dataset_summary: scans obs, SenMayo coverage, clusters, pipeline flags.

format_dataset_context: short text for first-turn system prompt.

16. Example request flow

User: "What is the p-value for senescence increase in T cells?"

```
[code block]
Browser POST /chat
  → main.chat()
    → run_agent
      → ensure_pipeline
        → Gemini_function_call: test_senescence_difference
        → statistics.py: 4 mouse medians @ 3m vs 4 @ 24m, Mann-Whitney
        → _format_test_summary → reply
        → tool_calls logged with full JSON
  ← frontend_appends sessionToolRuns
```

If Gemini calls compare_across_age instead, formatter may clarify to use test_senescence_difference for score p-values.

17. Configuration

Variable	Effect
-----	-----
GEMINI_API_KEY	Required for chat + reports
GEMINI_MODEL	Default gemini-2.5-flash
DEFAULT_AGENT_ITERATIONS	Default 3
FULL_PIPELINE_AGENT_ITERATIONS	Default 5
VITE_API_URL	Frontend → backend URL

18. Layer responsibilities (summary)

Layer	Responsibility
-----	-----
AnnData + pipeline	Reproducible preprocessing once per dataset
Tools	All numbers and plots
tool_router	Bind Gemini args → Python functions
Formatters	Turn JSON → scientist-facing text
Gemini	Tool choice + optional prose
Frontend	Upload, chat, tool log for reports
Reports	Audit trail from tool_runs

19. Known limitations

- In-memory cache lost on server restart (disk upload survives).
- No job queue – long analyses block HTTP request.
- score_genes magnitude is for ranking within dataset, not calibrated units.
- When not using direct summary path, LLM may add biological interpretation beyond tool JSON (mitigated by prompt, not hard-blocked).

20. Key file paths

[code block]

```
backend/main.py
backend/agent/agent.py, pipeline.py, tool_router.py, tool_schema.py
backend/agent/system_prompt.py, report.py, cache.py
backend/tools/*.py
backend/dataset_paths.py
backend/data/uploads/
frontend/src/App.tsx, ChatPanel.tsx, Plots.tsx, config.ts
.env (GEMINI_API_KEY at repo root)
```

End of architecture guide.